

Modül 26

Real-Time Applications

WebSocket, SSE, Live Updates

Modül: 26 / 30

Ders Sayısı: 6 ders

Seviye: İleri

Versiyon: Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 26'ya Hoş Geldin

Modern uygulamalar gerçek zamanlı güncelleme bekler — chat, bildirim, dashboard, live tracking. Bu modülde WebSocket, Server-Sent Events (SSE), ve modern reactive patterns ile real-time uygulamalar yazacağız.

Öğrenecekler:

- ✓ WebSocket protokolü ve native API
- ✓ RxJS websocket() ile reactive WebSocket
- ✓ Socket.io client entegrasyonu
- ✓ Server-Sent Events (SSE)
- ✓ Reconnection ve error handling stratejileri
- ✓ Real-time signal-based state

WebSocket Basics

Browser native WebSocket API.

❏ WebSocket Nedir?

WebSocket: Client ve server arasında iki yönlü, persistent bağlantı. HTTP'den farklı: bir kez bağlanır, sürekli açık kalır, her iki taraf da anında mesaj gönderebilir.

⚖ HTTP vs WebSocket

Konu	HTTP	WebSocket
Connection	Request başına	Persistent
Direction	Client → Server	İki yönlü
Overhead	Header her seferinde	İlk handshake sonrası minimal
Use case	CRUD operations	Live updates, chat

❏ Native WebSocket API

```
@Injectable({ providedIn: 'root' })
export class WebSocketService {
  private socket: WebSocket | null = null;

  connect(url: string) {
    this.socket = new WebSocket(url);

    this.socket.onopen = () => {
      console.log('Connected');
    };

    this.socket.onmessage = (event) => {
      const data = JSON.parse(event.data);
      console.log('Message:', data);
    };

    this.socket.onerror = (error) => {
      console.error('Error:', error);
    };

    this.socket.onclose = () => {
      console.log('Disconnected');
    };
  }

  send(data: any) {
    if (this.socket?.readyState === WebSocket.OPEN) {
      this.socket.send(JSON.stringify(data));
    }
  }

  disconnect() {
    this.socket?.close();
  }
}
```

RxJS WebSocket()

Reactive WebSocket — daha güçlü, daha temiz.

❏ WebSocket() Subject

```
import { WebSocket, WebSocketSubject } from 'rxjs/webSocket';

@Injectable({ providedIn: 'root' })
export class ChatService {
  private socket$: WebSocketSubject<any> | null = null;

  connect() {
    this.socket$ = WebSocket({
      url: 'wss://chat.example.com/ws',
      openObserver: {
        next: () => console.log('Connected')
      },
      closeObserver: {
        next: () => console.log('Disconnected')
      }
    });
  }

  return this.socket$.asObservable();
}

sendMessage(message: ChatMessage) {
  this.socket$?.next(message);
}

disconnect() {
  this.socket$?.complete();
}
}
```

❏ Component'te Kullanım

```

@Component({...})
export class ChatComponent {
  private chat = inject(ChatService);

  messages = signal<ChatMessage[]>([]);

  constructor() {
    this.chat.connect()
      .pipe(takeUntilDestroyed())
      .subscribe({
        next: (msg: ChatMessage) => {
          this.messages.update(msgs => [...msgs, msg]);
        },
        error: (err) => console.error(err)
      });
  }

  send(text: string) {
    this.chat.sendMessage({ text, timestamp: Date.now() });
  }
}

```

❑ Auto Reconnect

```

import { retry, timer } from 'rxjs';

connect() {
  return websocket('wss://example.com/ws').pipe(
    retry({
      delay: (error, retryCount) => {
        const delay = Math.min(1000 * Math.pow(2, retryCount), 30000);
        console.log(`Reconnecting in ${delay}ms (attempt ${retryCount})`);
        return timer(delay);
      }
    })
  );
}

```

Socket.io Client

Production'da en çok kullanılan WebSocket library.

❏ Kurulum

```
npm install socket.io-client  
  
# TypeScript types otomatik gelir
```

❏ Socket.io Service

```

import { io, Socket } from 'socket.io-client';

@Injectable({ providedIn: 'root' })
export class SocketIoService {
  private socket: Socket | null = null;

  connect(url: string, options?: any) {
    this.socket = io(url, {
      transports: ['websocket'],
      reconnection: true,
      reconnectionAttempts: 5,
      reconnectionDelay: 1000,
      ...options
    });

    this.socket.on('connect', () => {
      console.log('Connected:', this.socket?.id);
    });

    this.socket.on('disconnect', (reason) => {
      console.log('Disconnected:', reason);
    });
  }

  on<T>(event: string): Observable<T> {
    return new Observable<T>(subscriber => {
      const handler = (data: T) => subscriber.next(data);
      this.socket?.on(event, handler);

      return () => this.socket?.off(event, handler);
    });
  }

  emit(event: string, data?: any) {
    this.socket?.emit(event, data);
  }

  disconnect() {
    this.socket?.disconnect();
  }
}

```

□ Use Case: Notifications


```

@Component({...})
export class NotificationCenter {
  private socketIo = inject(SocketIoService);
  private toast = inject(MessageService);

  notifications = signal<Notification[]>([]);

  constructor() {
    this.socketIo.connect('https://api.example.com', {
      auth: { token: this.auth.getToken() }
    });

    // Listen for notifications
    this.socketIo.on<Notification>('notification')
      .pipe(takeUntilDestroyed())
      .subscribe(notif => {
        this.notifications.update(n => [notif, ...n]);
        this.toast.add({
          severity: 'info',
          summary: notif.title,
          detail: notif.message
        });
      });

    // Listen for user-specific events
    this.socketIo.on('user-update')
      .pipe(takeUntilDestroyed())
      .subscribe(update => {
        // Handle
      });
  }

  markAsRead(id: string) {
    this.socketIo.emit('mark-read', { id });
  }
}

```

Server-Sent Events (SSE)

WebSocket'in basit alternatifi — sadece server → client.

☐ SSE Nedir?

SSE: Server → Client tek yönlü stream. HTTP üzerinden çalışır, basit, otomatik reconnect. WebSocket gerekmediğinde tercih edilir (notification, log streaming).

⚖️ WebSocket vs SSE

Konu	WebSocket	SSE
Yön	İki yönlü	Server → Client
Protocol	ws://	http://
Format	Binary/text	Text only
Reconnect	Manuel	Otomatik
Karmaşıklık	Yüksek	Düşük

☐ EventSource API

```

@Injectable({ providedIn: 'root' })
export class SseService {
  private zone = inject(NgZone);

  connect<T>(url: string): Observable<T> {
    return new Observable<T>(observer => {
      const source = new EventSource(url, { withCredentials: true });

      source.onmessage = event => {
        this.zone.run(() => {
          try {
            const data = JSON.parse(event.data);
            observer.next(data);
          } catch {
            observer.next(event.data as any);
          }
        });
      };

      source.onerror = error => {
        this.zone.run(() => observer.error(error));
      };

      return () => source.close();
    });
  }
}

```

□ Use Case: Live Logs

```

@Component({
  template: `
    <div class="log-stream">
      @for (log of logs(); track log.id) {
        <div [class]="`log-${log.level}`">
          <span>{{ log.timestamp | date:'HH:mm:ss' }}</span>
          <span>{{ log.message }}</span>
        </div>
      }
    </div>
  `,
})
export class LogStreamComponent {
  private sse = inject(SseService);
  logs = signal<LogEntry[]>([]);

  constructor() {
    this.sse.connect<LogEntry>('/api/logs/stream')
      .pipe(takeUntilDestroyed())
      .subscribe(log => {
        this.logs.update(arr => [...arr.slice(-100), log]); // Son 100
      });
  }
}

```

□ Custom Event Types

```

connect<T>(url: string, events: string[]): Observable<{ type: string; data: T }> {
  return new Observable(observer => {
    const source = new EventSource(url);

    events.forEach(eventType => {
      source.addEventListener(eventType, (event: any) => {
        observer.next({
          type: eventType,
          data: JSON.parse(event.data)
        });
      });
    });

    return () => source.close();
  });
}

// Backend gönderir:
// event: notification
// data: {"id": "1", "title": "Hello"}
//
// event: status-update
// data: {"online": true}

```

Reconnection & Error Handling

Production-grade resilience.

❑ Exponential Backoff

```
import { Subject, retry, timer, EMPTY } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class ResilientWebSocketService {
  private destroy$ = new Subject<void>();
  isConnected = signal(false);
  reconnectAttempt = signal(0);

  connect(url: string) {
    return websocket(url).pipe(
      tap({
        subscribe: () => this.isConnected.set(true),
        unsubscribe: () => this.isConnected.set(false),
      }),
      retry({
        delay: (error, attempt) => {
          this.reconnectAttempt.set(attempt);

          if (attempt > 10) {
            console.error('Max reconnect attempts');
            return EMPTY;
          }

          // Exponential backoff: 1s, 2s, 4s, 8s, ... max 30s
          const delay = Math.min(1000 * Math.pow(2, attempt - 1), 30000);
          console.log(`Reconnecting in ${delay}ms (attempt ${attempt})`);
          return timer(delay);
        }
      }),
      tap({
        next: () => this.reconnectAttempt.set(0), // Reset on success
      })
    );
  }
}
```

❑ Heartbeat / Keep-Alive

```

import { interval } from 'rxjs';

@Injectable({...})
export class HeartbeatWebSocketService {
  private socket$: WebSocketSubject<any> | null = null;

  connect(url: string) {
    this.socket$ = websocket(url);

    // Her 30 saniyede ping gönder
    interval(30000)
      .pipe(takeUntilDestroyed())
      .subscribe(() => {
        if (this.socket$ && !this.socket$.closed) {
          this.socket$.next({ type: 'ping' });
        }
      });

    return this.socket$.asObservable();
  }
}

```

□ Connection State Tracking

```

type ConnectionState = 'connecting' | 'connected' | 'disconnected' | 'error';

@Injectable({ providedIn: 'root' })
export class ConnectionTracker {
  state = signal<ConnectionState>('disconnected');

  setConnecting() { this.state.set('connecting'); }
  setConnected() { this.state.set('connected'); }
  setDisconnected() { this.state.set('disconnected'); }
  setError() { this.state.set('error'); }
}

// Component
@if (connection.state() === 'connecting') {
  <p>□ Bağlanıyor...</p>
}
@if (connection.state() === 'disconnected') {
  <p class="error">△ Bağlantı kesildi</p>
}

```

Real-Time Signal Pattern

Modern signal-based real-time state.

⚡ Signal-Based Live Counter

```
@Injectable({ providedIn: 'root' })
export class LiveDashboardService {
  private socket$ = websocket<DashboardUpdate>('wss://api.example.com/dashboard');

  // Signal-based state
  activeUsers = signal(0);
  revenue = signal(0);
  orders = signal<Order[]>([]);

  constructor() {
    this.socket$.pipe(
      takeUntilDestroyed()
    ).subscribe(update => {
      switch (update.type) {
        case 'user-count':
          this.activeUsers.set(update.value);
          break;

        case 'revenue':
          this.revenue.set(update.value);
          break;

        case 'new-order':
          this.orders.update(o => [update.order, ...o.slice(0, 99)]);
          break;
      }
    });
  }
}
```

□ Live Dashboard Component

```

@Component({
  template: `
    <div class="dashboard">
      <div class="stat">
        <h3>{{ dashboard.activeUsers() }}</h3>
        <p>Aktif Kullanıcı</p>
      </div>

      <div class="stat">
        <h3>{{ dashboard.revenue() | currency }}</h3>
        <p>Gelir</p>
      </div>

      <div class="orders">
        <h3>Son Siparişler</h3>
        @for (order of dashboard.orders(); track order.id) {
          <div class="order order-new">
            {{ order.customer }} – {{ order.total | currency }}
          </div>
        }
      </div>
    </div>
  `,
  styles: [
    .order-new {
      animation: pulse 2s ease-out;
    }
    @keyframes pulse {
      0% { background: yellow; }
      100% { background: transparent; }
    }
  ]
})
export class DashboardComponent {
  protected dashboard = inject(LiveDashboardService);
}

```

□ Best Practices

- ✓ takeUntilDestroyed() ile auto cleanup
- ✓ Signal-based state ile reactive UI
- ✓ Reconnection logic production'da olmalı
- ✓ Connection state kullanıcıya göster
- ✓ Message buffering reconnect sırasında
- ✓ Auth token connect sırasında gönder
- ✓ Rate limiting client-side (server'ı koru)

☐ Modül 26 Özeti

Real-time uygulamalar artık çocuk oyuncağı. WebSocket, SSE, Socket.io — hangisini ne zaman kullanacağını biliyorsun. Reconnection, error handling, signal-based state — production hazır.

☐ Neler Öğrendin?

- ✓ WebSocket native API
- ✓ RxJS websocket() reactive yaklaşım
- ✓ Socket.io client
- ✓ Server-Sent Events (SSE)
- ✓ Reconnection ve exponential backoff
- ✓ Real-time signal-based state pattern

☐ Sıradaki Modül

Modül 27 — File Upload & Download. Multipart, progress tracking, chunked upload, streaming downloads.